UD1 – Persistència en Bases de dades documentals. MongoDB



ÍNDEX

Índex de continguts

1 Bases de dades NoSQL....5

1. Estructura d'una BD MongoDB....6

1.1 Col·leccions i Documents....6
Col·lecció:....7
Document:....7

1.2 Camp _id....9

1.3 Esquema de dades - Relacions en el model....11
One-to-one....11
One-to-many incrustat....11
One-to-many amb referències....11
1.3.1 One-to-one (UN a UN)....11
1.3.2 One-to-many incrustat (UN a MOLTS)....11
1.3.3 One-to-many amb referències (UN a MOLTS)....13

2 Instal·lació del Sistema Gestor de Bases de Dades MongoDB....14

3 Connexió amb MongoDB Compass al servidor de bases de dades....18

3.1 Creació d'una base de dades i una col·lecció....19
pel·lícules....19

3.2 Importació de dades d'un document JSON....21

4 Connexió a MongoDB desde Java....24

4.1 Creació i configuració projecte Java amb connexió amb MongoDB....24
1.- File New Project Maven Maven Project....24

4.2 Creem la classe "ObtindrePeliculesMongoDB"....27
4.2.1 Llibreries de classes per a importar....29

4.3 Establint una connexió....30
4.3.1 Connectar-se a una única instància de MongoDB....30

4.4 Obtenint una Base de dades....31

4.5 Obtenint una Col·lecció....32

5 Consultes a la base de dades MongoDB....33

5.1 Classe Document....33
Exemple 1 des de Java:....33
Exemple 2: Obtindre valors per al document des d'un Pojo:....33
5.2 Obtindre el primer document d'una col·lecció....34
5.3 Obtindre tots els documents d'una col·lecció....36
5.4 Classe Filters: especificar un filtre de consulta....37
5.4.1 Obtindre un document únic que coincidisca amb un filtre....39
5.4.2 Obtindre tots els documents que coincideixen amb un filtre....40
Exemple 1:....40
5.5 Contar documents en una col·lecció....41
5.6 Buscar patrons en els camps dels documents....41
Exemple 3:....41
Exemple 4:....43
Exemple 5:....43
5.7 Consultes sobre arrays....43
5.7.1 Buscar elements dins d'un array....43
Exemple 6:....43
5.7.2 Contar els elements que conté un array....44
Exemple 7:....44
5.8 Projeccions....45
1. Especificant literalment com volem que siga el document resultant:....45
5.9 Ordenant els resultats....46
Exemple 8:....46
6 Inserció de documents a la base de dades MongoDB....47
6.1 Crear un document....47
6.2 Inserir un Document....48
6.3 Inserir Múltiples Documents....48
7 Actualització de documents a la base de dades MongoDB....50
7.1 Actualitzar un únic document....50
7.2 Actualitzar múltiples documents....51
8 Eliminació de documents a la base de dades MongoDB....51
8.1 Eliminar un únic document que compleix una condició....51

8.2 Eliminar tots els documents que compleixen una condició.....52

1 Bases de dades NoSQL

A pesar que les bases de dades relacionals han satisfet les necessitats de la immensa majoria d'usuaris en les últimes dècades, no podríem dir que són infal·libles en totes les solucions.

Si bé la quota de mercat de les solucions NoSQL no està a l'altura de les relacionals, les majors empreses d'Internet ja tenen els seus propis sistemes basats en esta tecnologia: Amazon amb la seua **DinamoDB** o Linkedin amb **RedIs**.

El contingut d'este tema que comencem girarà entorn a una d'estes bases de dades NoSQL, concretament de la base de dades documental MongoDB.



1. Estructura d'una BD MongoDB

Si tenim moltes dades no estructurades i de les quals es realitzaran moltes consultes, però poques modificacions, el model del document és una bona opció.

Per exemple, les estadístiques de trànsit d'un lloc web, xarxes socials, publicitat… Emmagatzemar gran quantitat d'informació, feblement relacionada i que no és necessària la consistència del 100%.

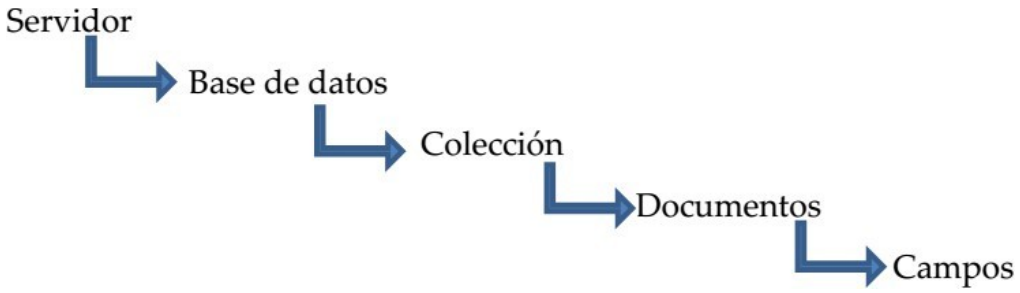
MongoDB és una **Base de dades NoSQL orientada a documents** en format **BSON** Binary Javascript Object Notation), similar a JSON. Este és el format que s'utilitza intercanvi de dades per a emmagatzematge i transferència de documents en MongoDB. Es tracta d'una representació binària d'estructures de dades i mapes, dissenyada per a ser més lleuger i eficient que JSON, (Javascript Object Notation), amb algunes **particularitats**:

- Tipus de dades addicionals.
- Dissenyat per a ser més eficient en espai, encara que a vegades un JSON pot ocupar menys que un BSON.
- Dissenyat perquè les cerques siguen més eficients.

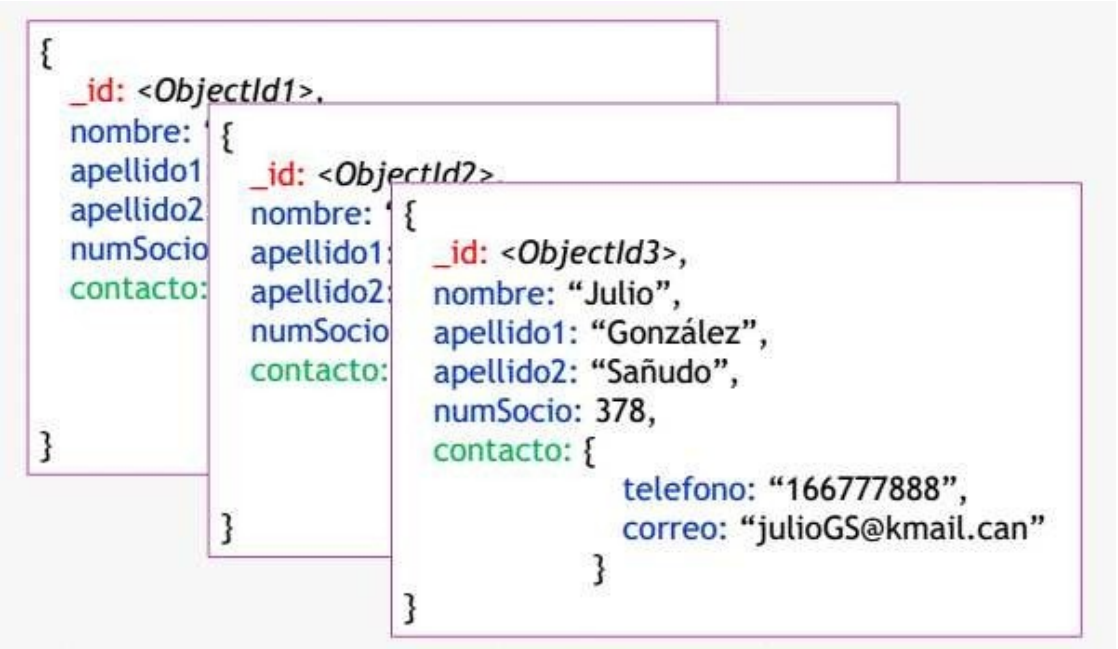
1.1 Col·leccions i Documents

Les bases de dades estan formades per **Col·leccions**, i al mateix temps, cada servidor pot tindre tantes bases de dades com l'equip ho permeta.

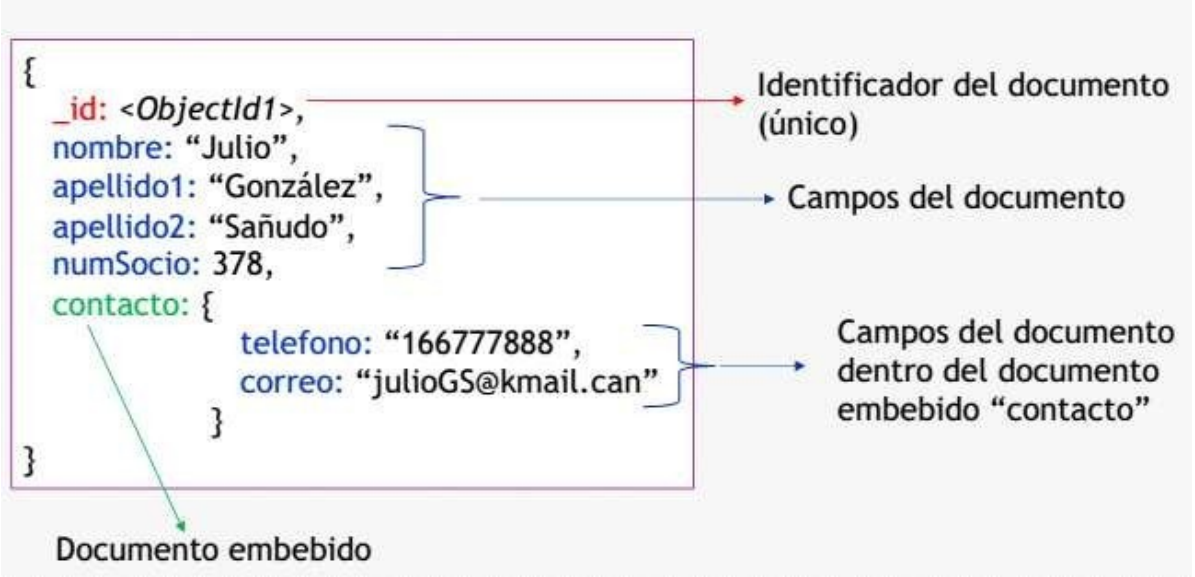
En MongoDB, cada registre o conjunt de dades es denomina **document**, que poden ser agrupats en **col·leccions**, (equivalent a les taules de les bases de dades relacionals però sense estar sotmesos a un esquema fix).



Col·lecció:



Document:



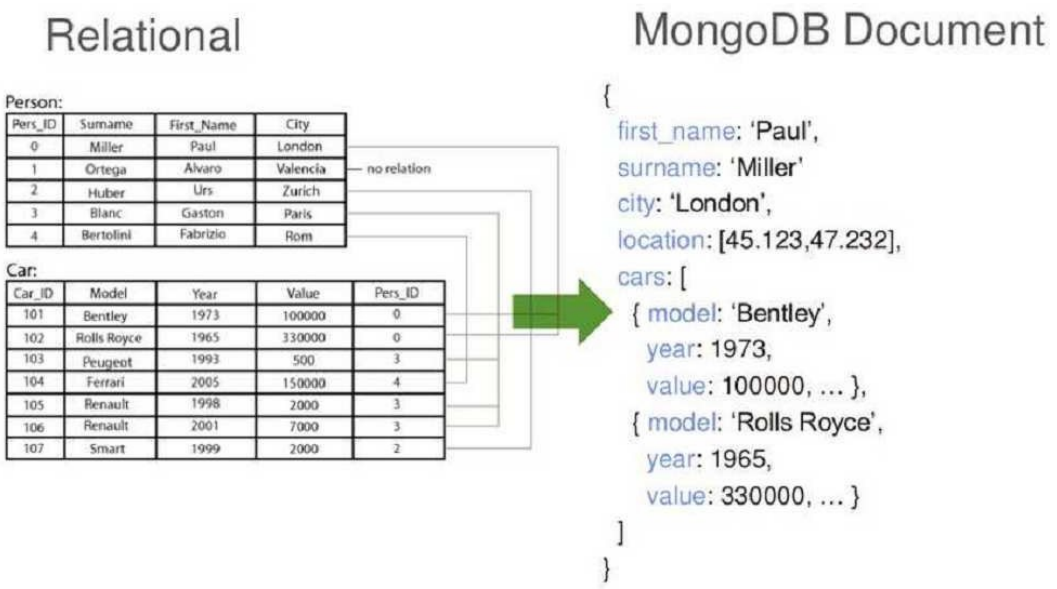
Un **registre** de MongoDB és un document, una estructura de dades composta per **parells de camps i de valor**. Estos documents són similars als objectes **JSON**. Els valors dels camps poden incloure:

- Altres documents.
- Matrius (arrays).

- Conjunts de documents.

En MongoDB, documents emmagatzemats en una col·lecció han de tindre un **camp _id únic,** que actua com una clau principal.

Un conjunt de Camps formaran un Document, que en cas d'associar-se amb uns altres formarà una Col·lecció.



La jerarquia principal de MongoDB ve donada pels elements presentats a dalt. La diferència amb els SGBDR, (Sistemes Gestors de Bases de dades Relacionals), és que no utilitzem taules, files ni columnes com allí ho féiem, sinó que utilitzem documents amb diferents estructuresEn MongoDB, com ja s'ha comentat adés, no existeix un esquema estàndard per a treballar amb les dades, però això no significa que anem a tindre una quantitat ingent de dades difícils de relacionar.

De fet, la majoria de les vegades treballarem amb documents estructurats, només que no seguiran el mateix esquema tots ells, sinó que cadascun podrà tindre un propi si resulta apropiat treballar amb ells.

Imaginem que tenim una col·lecció a la qual anomenem Persones. Un document podria a emmagatzemar-se la següent manera:

```
{
  Nom: "Javi",
  Cognoms: "Martínez Campos",
  Edat: 22,
  Aficions: ["futbol","tennis","ciclisme"],
  Amics: [
   { Nom:"Maria",
     Edat:22    },
   { Nom:"Lluis",
     Edat:28    }
  ]
}
```

- Les **claus {}** delimiten documents embeguts és a dir, un conjunt de parells clau - valor.
- Els **[]** delimiten un array, llista de valors associats a una clau.

El document anterior és un clàssic document JSON. Té strings, arrays, subdocuments i números. En la mateixa col·lecció podríem guardar un document com este:

```
{
  Nom: "Luis",
  Estudis: "Administració i Direcció d'Empreses",
  Amics:12
}
```

Este document no seguix el mateix esquema que el primer. Té menys camps, algun camp nou que no existix en el document anterior i fins i tot un camp de diferent tipus.

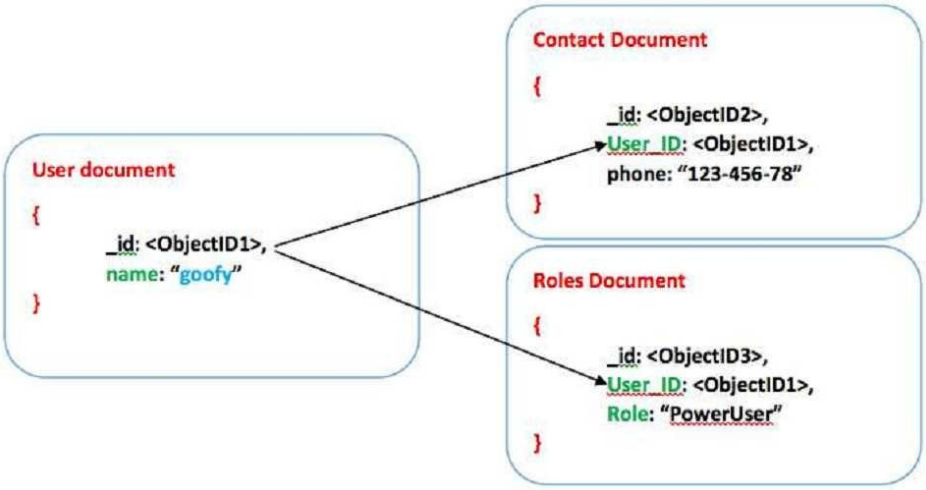
Açò, que seria impensable en una base de dades relacional, és una cosa totalment vàlida en MongoDB.

1.2 Camp _id

El camp especial _id, creat amb ObjectId. Este objecte, que és en realitat tractat com un String, es crea automàticament en tots els documents que inserim en la nostra base de dades i té un valor únic.

- Podem especificar nosaltres el valor que vulguem, però haurem de tindre en compte que **no pot haver dos documents amb el mateix _id**.
- **El camp _id es crea automàticament amb la funció ObjectId**, però podem inserir valors únics generats amb esta funció en qualsevol camp del nostre document.
- **_id** és un número hexadecimal de 12 bytes únic per a cada document en una col·lecció.
- L'ID dels registres és preferible introduir-lo manualment perquè si deixem que el propi MongoDB ens l'autogenere no ho fa com un número enter normal i a l'hora de buscar-lo des d'una aplicació pot resultar complicat.

Així que es recomanable generar els ID dels registres en el servidor amb un generador de ids únic i afegir-lo als teus registres.



1.3 Esquema de dades - Relacions en el model

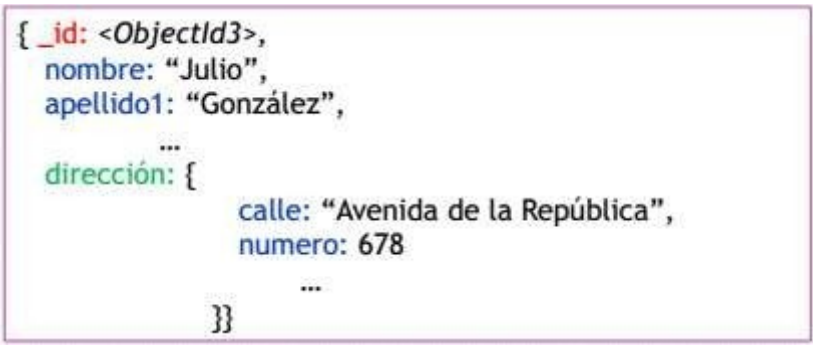
Com hem dit adés, en MongoDB l'esquema de les dades és flexible. Correspon al desenvolupador decidir cóm implementar-lo segons els requisits que es tinguen.

Es poden definir 3 **tipus de relacions** que vorem a continuació.

- **One-to-one**
- **One-to-many incrustat**
- **One-to-many amb referències**

1.3.1 One-to-one (UN a UN)

En este tipus de relació, un dels documents sol incloure's dins d'un altre. Imaginem que emmagatzemem les dades dels socis d'un gimnàs juntament amb la seua direcció, que es compon dels seus propis camps. La solució passaria per incloure la direcció com un subdocument del document principal.



1.3.2 One-to-many incrustat (UN a MOLTS)

Els documents de la relació s'inclouen dins d'un altre en una estructura de tipus array. Seguint amb l'exemple anterior, imaginem que els socis poden tindre diverses direccions:

```
{ _id: <ObjectId3>,
  nombre: "Julio",
  apellido1: "González",
          ...
  direcciones: [{
                  calle: "Avenida de la República",
                  numero: 678
                      ...},
                 {calle: "Avenida de la República",
                  numero: 678
                      ... }]}
```

A voltes, pot ser molt pesat emmagatzemar matrius (arrays) amb totes les dades dels subdocuments. Imaginem que volem emmagatzemar llibres juntament amb la referència als editors (molts).

Si pensem en un exemple en el qual podríem emmagatzemar informació sobre llibres d'un editor:

a) Una opció seria els llibres en una matriu (array) com subdocuments dels documents dels editors:

```
{ _id: "edicionesSotileza",
  ciudad: "Santander",
          ...
  libros: [{
            nombre: "Historia de Cantabria",
            ISBN: "678789789"
                    ...},
            nombre: "El pleito de los nueve valles",
            ISBN: "2671982093"
                    ...
```

El problema de l'anterior solució és que la matriu (array) podria créixer en excés, amb el que les cerques seria molt ineficients.

b) Podríem pensar com a solució incloure als editors en els llibres:



El problema de l'anterior solució és que es repetirien amb freqüència les dades dels editors, ocupant una gran quantitat d'espai.

1.3.3 One-to-many amb referències (UN a MOLTS)

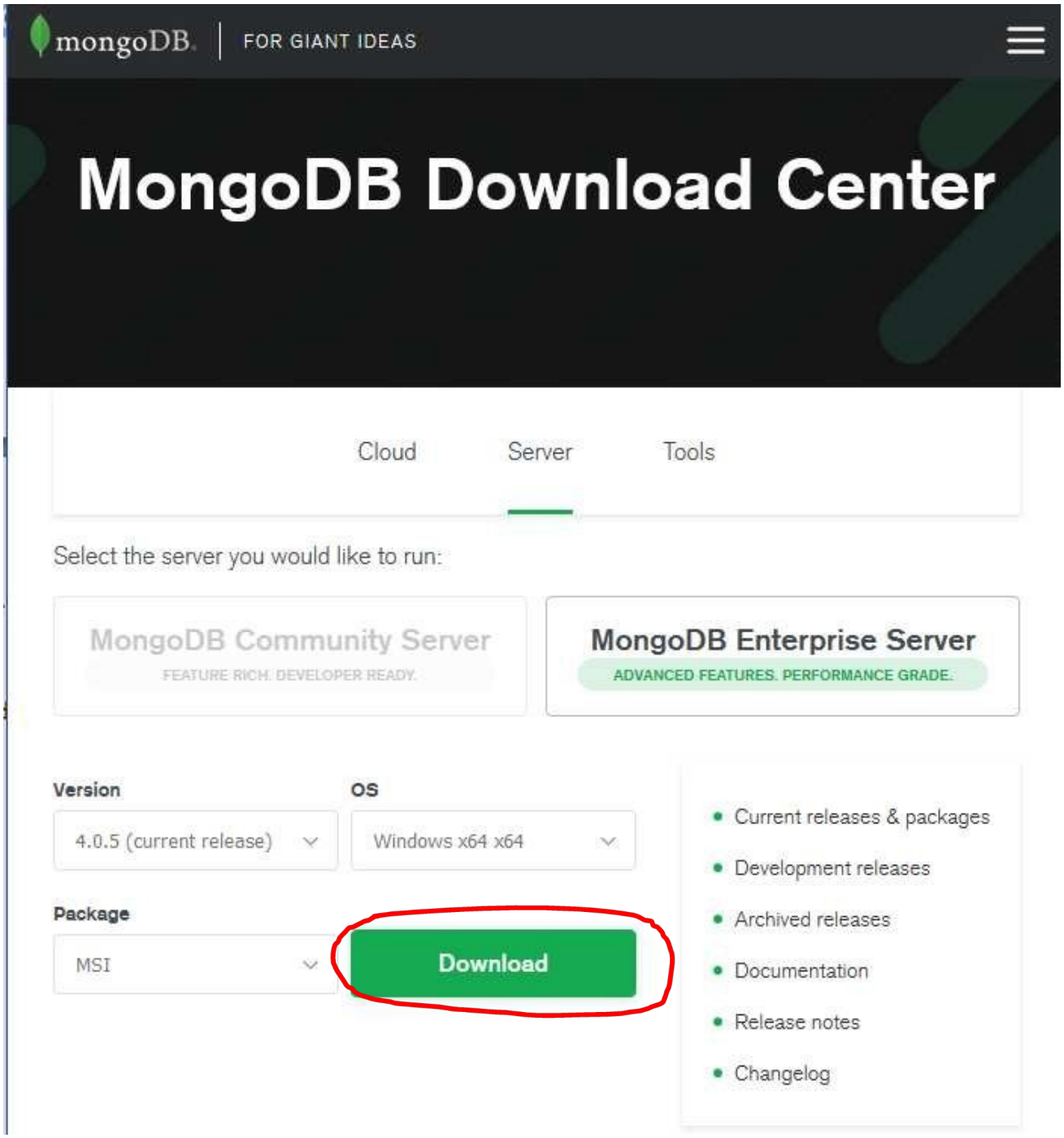
S'utilitzen referències al _id dels documents relacionats, en comptes d'incloure'ls per complet.



2 Instal·lació del Sistema Gestor de Bases de Dades MongoDB.

En primer lloc, caldrà anar a la pàgina oficial de MongoDB per tal de descarregar-nos el servidor MongoDB en la seua versió Enterprise (descarregueu-vos la versió 4.0)

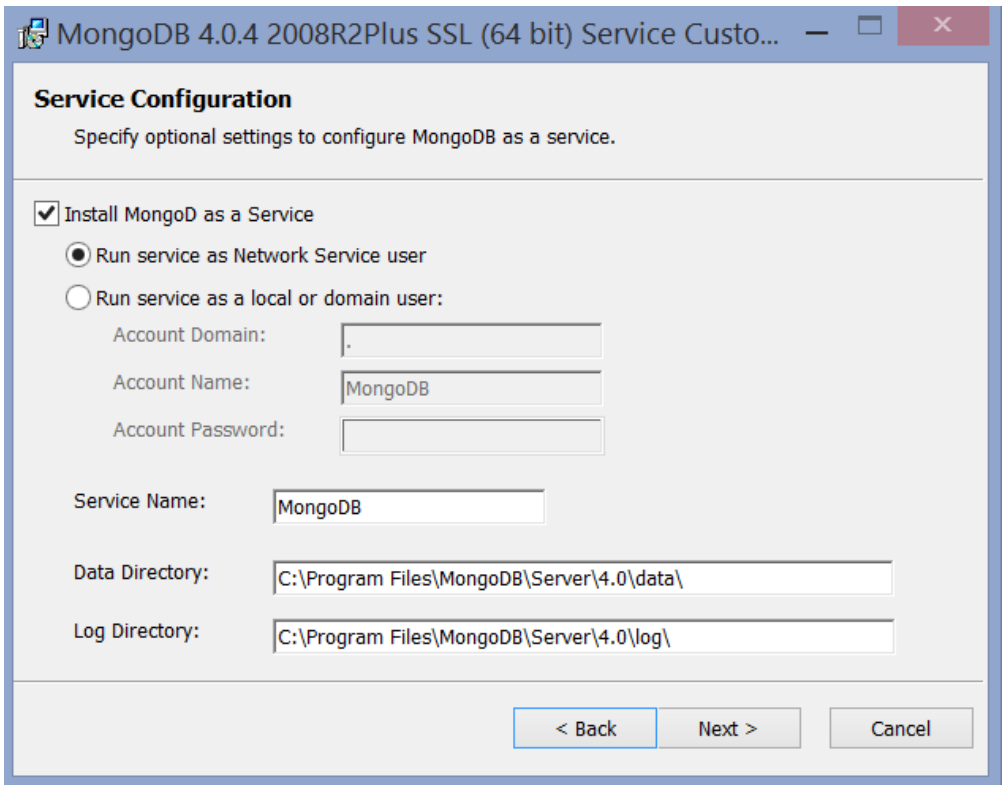
https://www.mongodb.com/download-center/enterprise



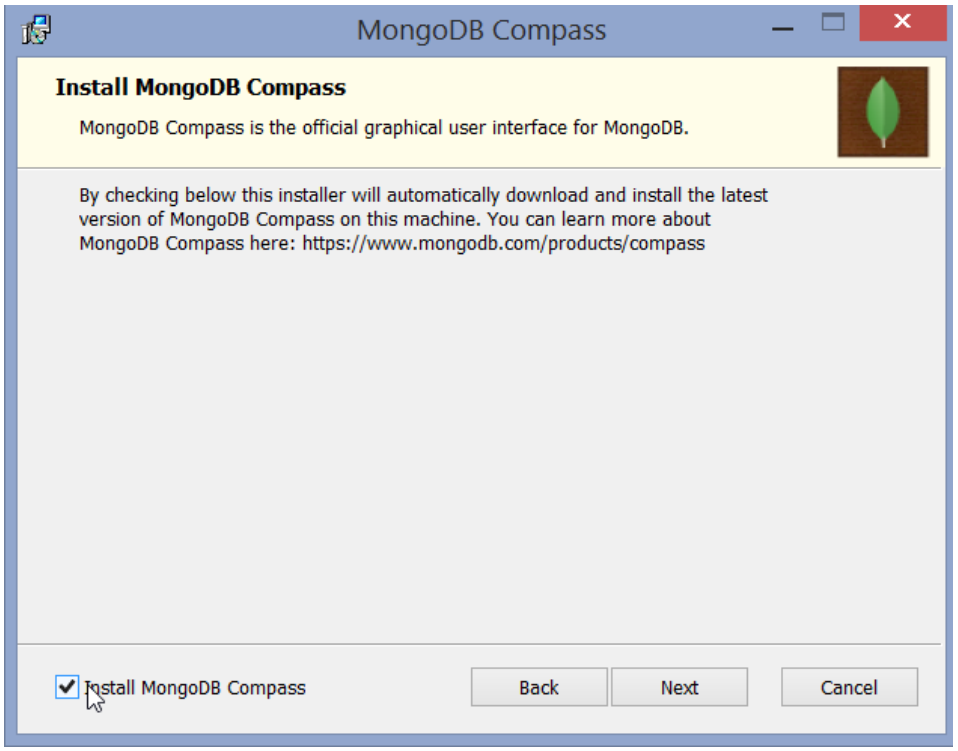
Un volta descarregat el paquet MSI (Microsoft Windows Installer), l'executem com a Administradors del sistema i continuem amb el procés d'instal·lació. Hem de triar la instal·lació completa per disposar de totes les funcionalitats que té esta versió.

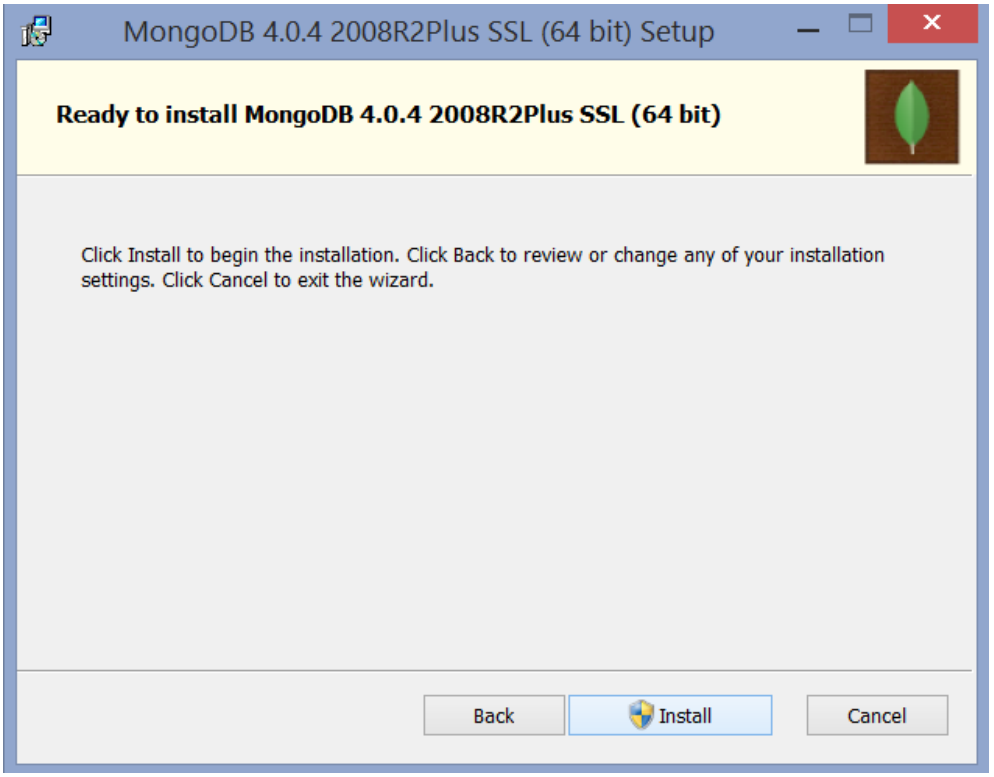


En aplegar a la pantalla on es tria la configuració de MongoDB, cal triar l'opció que l'instal·larà com a servici deixant totes les opcions per defecte.



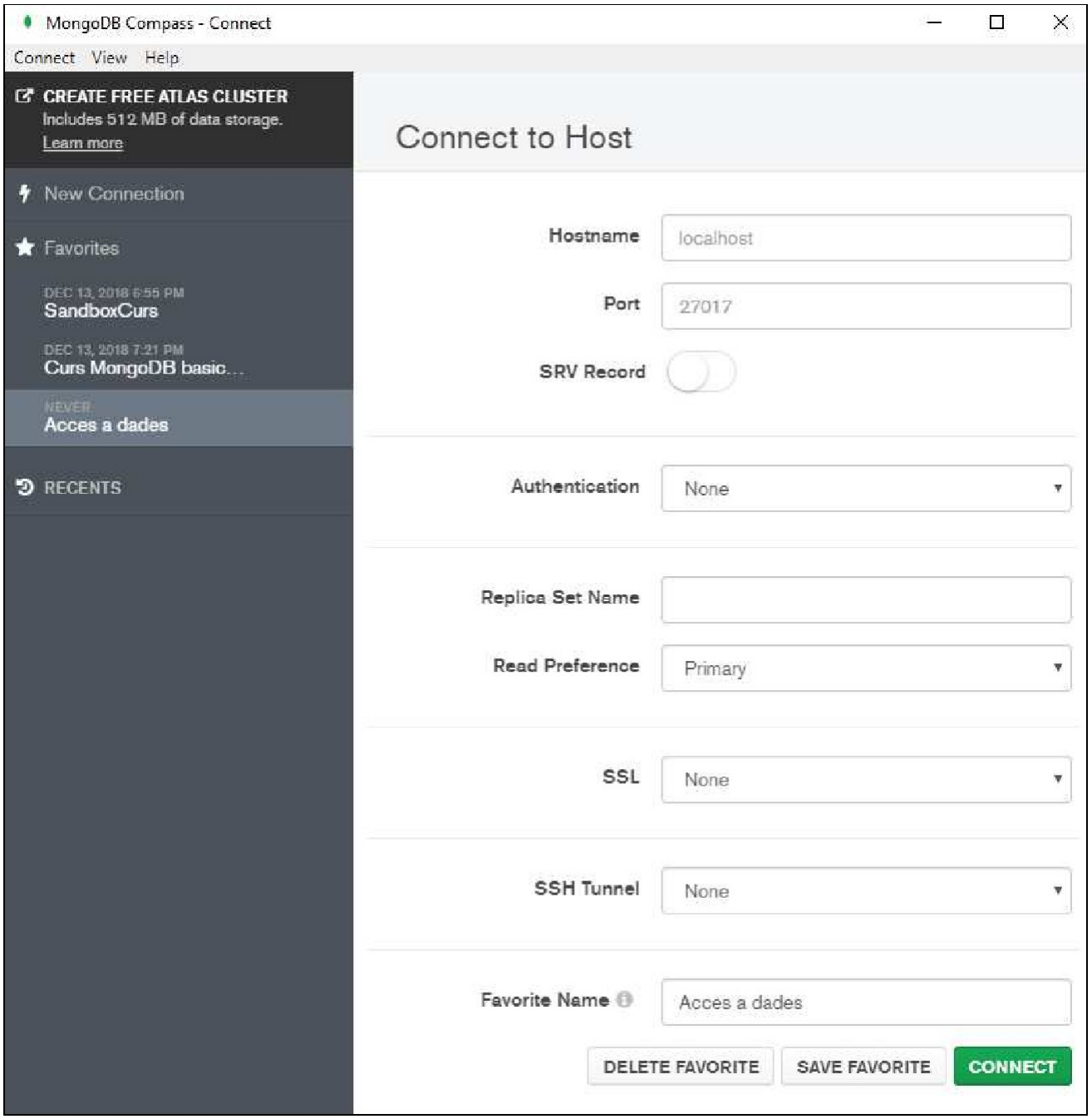
Durant el procés d'instal·lació, l'assistent ens preguntarà si volem instal·lar l'aplicació **MongoDB Compass** (interfície gràfica d'usuari oficial de MongoDB). Marquem la casella per tal que ho faça.


MongoDB Compass
Install MongoDB Compass
MongoDB Compass is the official graphical user interface for MongoDB.
By checking below this installer will automatically download and install the latest version of MongoDB Compass on this machine. You can learn more about MongoDB Compass here: https://www.mongodb.com/products/compass
Install MongoDB Compass
Back
Next
Cancel


MongoDB 4.0.4 2008R2Plus SSL (64 bit) Setup
Ready to install MongoDB 4.0.4 2008R2Plus SSL (64 bit)
Click Install to begin the installation. Click Back to review or change any of your installation settings. Click Cancel to exit the wizard.
Back
Install
Cancel

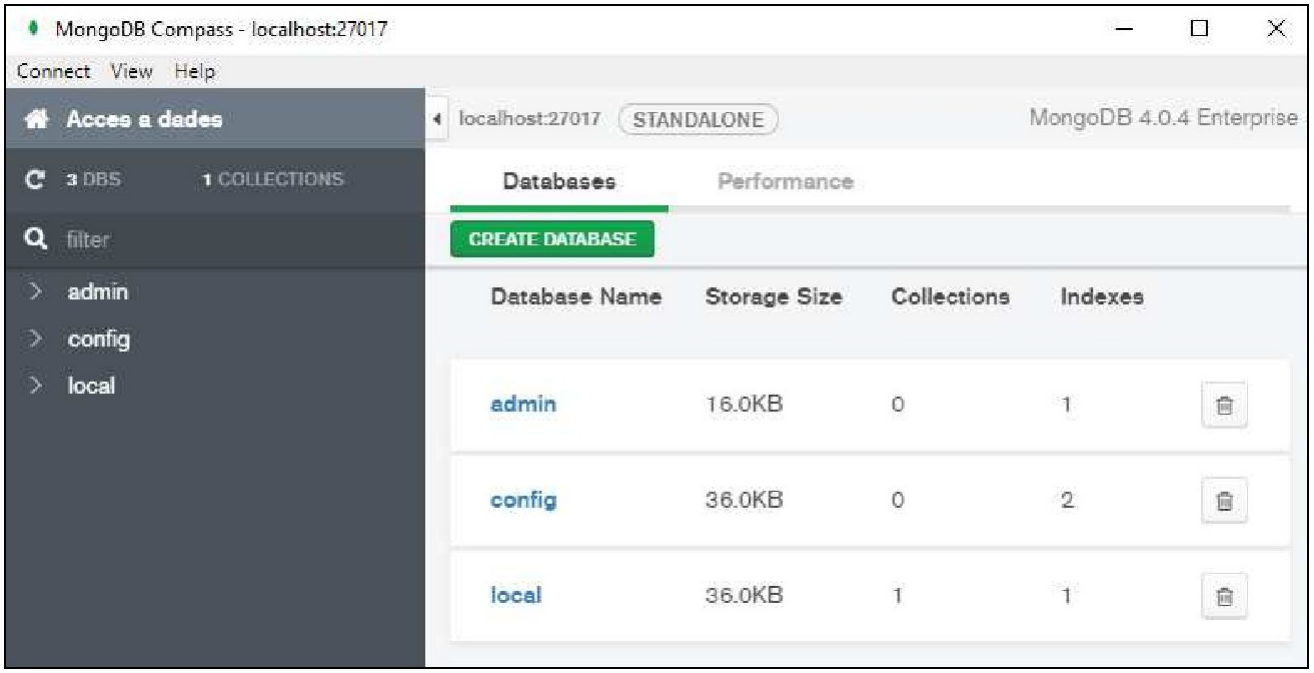
3 Connexió amb MongoDB Compass al servidor de bases de dades

Obrim l'aplicació que acabem d'instal·lar en el punt anterior (MongoDB Compass) i obtindrem la següent pantalla:



En el camp “Favorite Name” posem el nom “**Accés a dades**” i polsem el botó “SAVE FAVORITE”, de manera que cada volta que entrem amb el Compass tinguem un accés directe a la connexió al servidor MongoDB que tenim instal·lat en el nostre equip.

Una volta creat, polsem en “CONNECT” i ens durà a la interfície del nostre servidor de bases de dades:



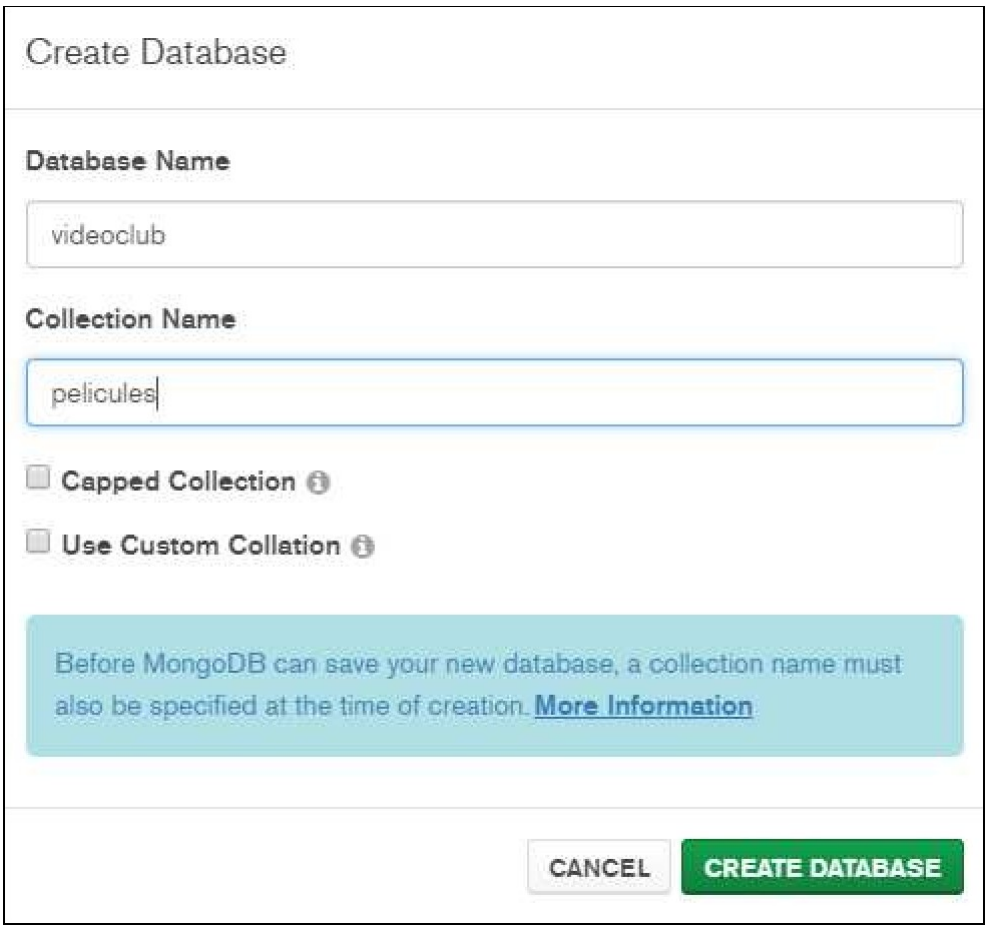
En esta vista podem vore les Bases de Dades que es creen per defecte al servidor MongoDB:

- admin
- config
- local

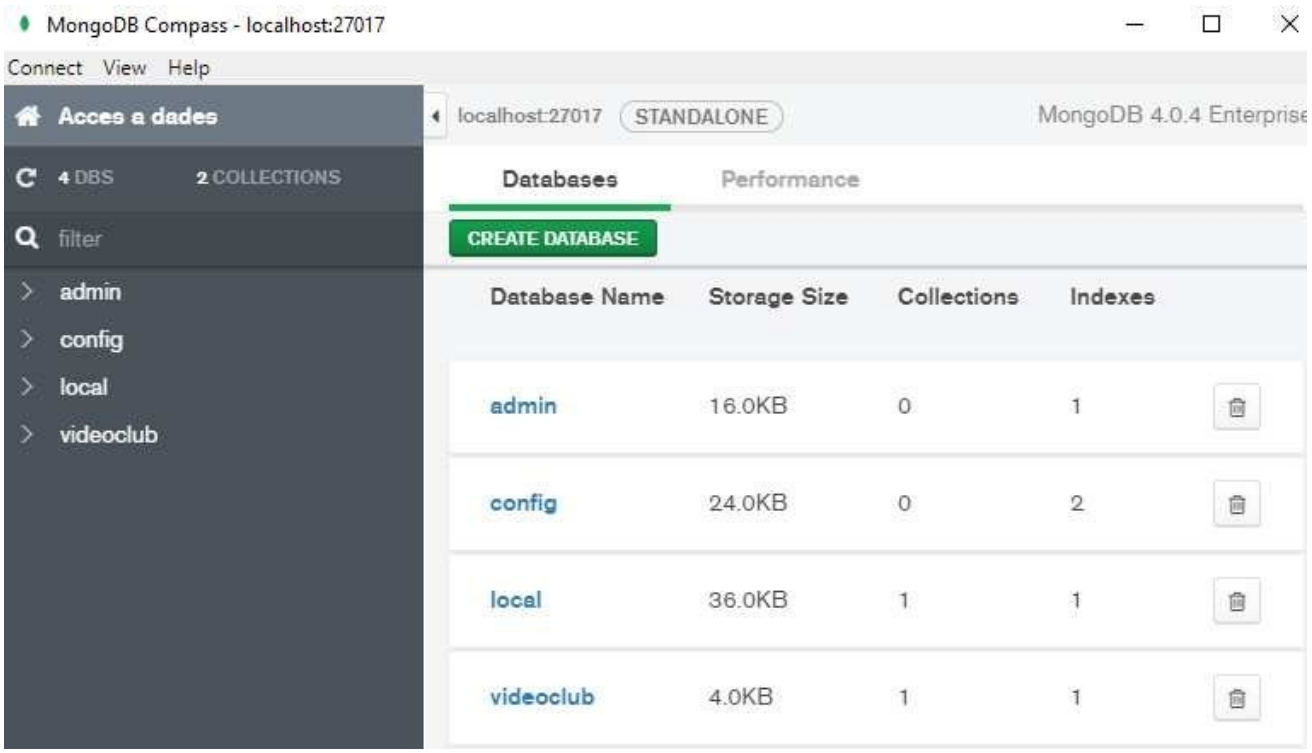
3.1 Creació d’una base de dades i una col·lecció

En el cas de les nostres pràctiques, anem a utilitzar una base de dades de **pel·lícules**.

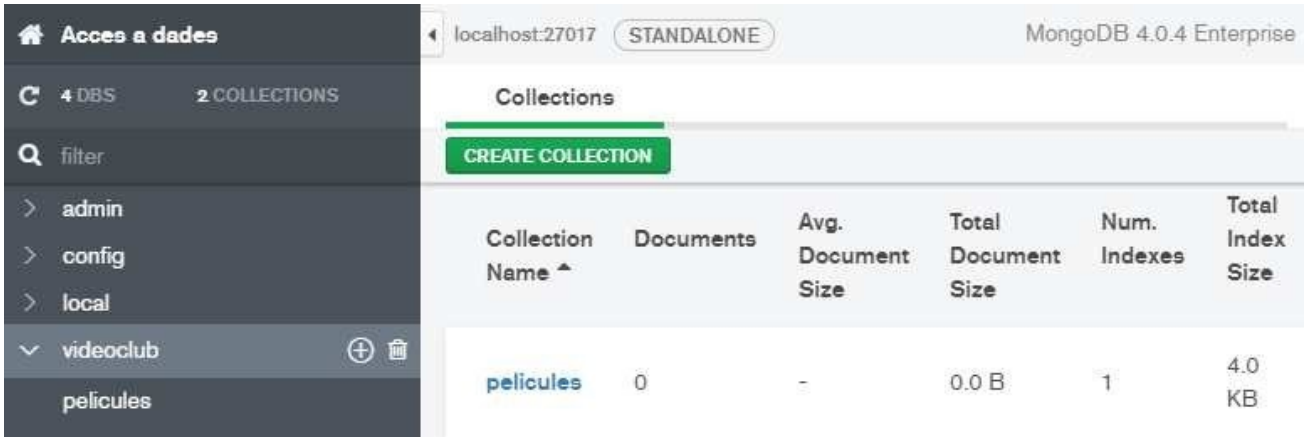
Per a crear la nostra pròpia base de dades, polsarem el botó “CREATE DATABASE” i posem el nom de la BD i de la primera col·lecció que volem crear.



Com podem apreciar, la nostra base de dades “videoclub” ja es troba al servidor local.



Així com també la nostra primera col·lecció “pelicules”:



3.2 Importació de dades d’un document JSON

Una volta creada la base de dades i la primera col·lecció, cal omplir-la amb dades. N’hi ha varies formes de fer-ho. En este moment anem a utilitzar la ferramenta “**mongoimport**” que s’instal·la junt amb el Compass.

El primer pas serà col·locar en la carpeta on estan els fitxers binaris de MongoDB el fitxer JSON que volem importar. Generalment, a Windows serà la següent carpeta:

C:\Program Files\MongoDB\Server\4.0\bin

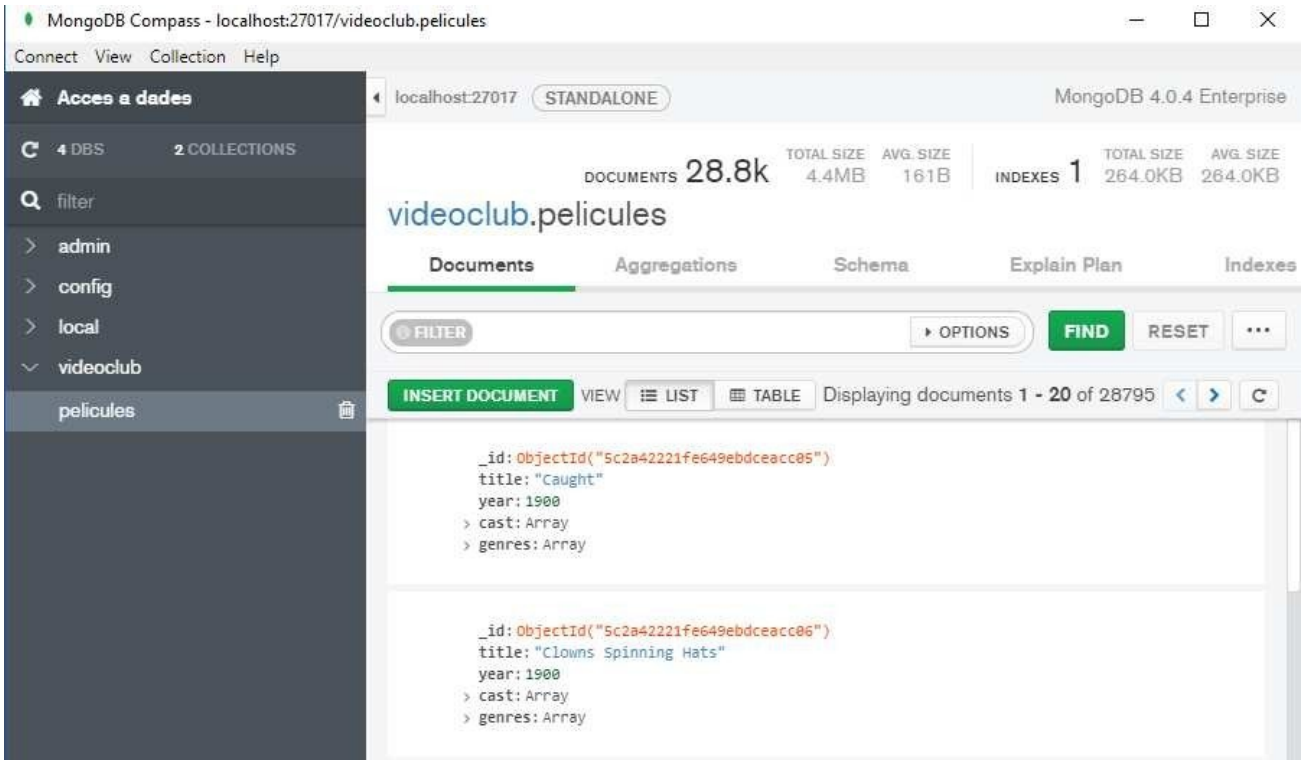
A continuació, haurem d'obrir la consola d'ordres de Windows (CMD) i situar-nos en eixa carpeta. Des d'esta ubicació, caldrà executar la següent ordre:

```
C:\Program Files\MongoDB\Server\4.0\bin>mongoimport --jsonArray
--db videoclub --collection pelicules --file movies.json
2018-12-31T17:21:54.285+0100    connected to: localhost
2018-12-31T17:21:54.886+0100    imported 28795 documents

C:\Program Files\MongoDB\Server\4.0\bin>
```

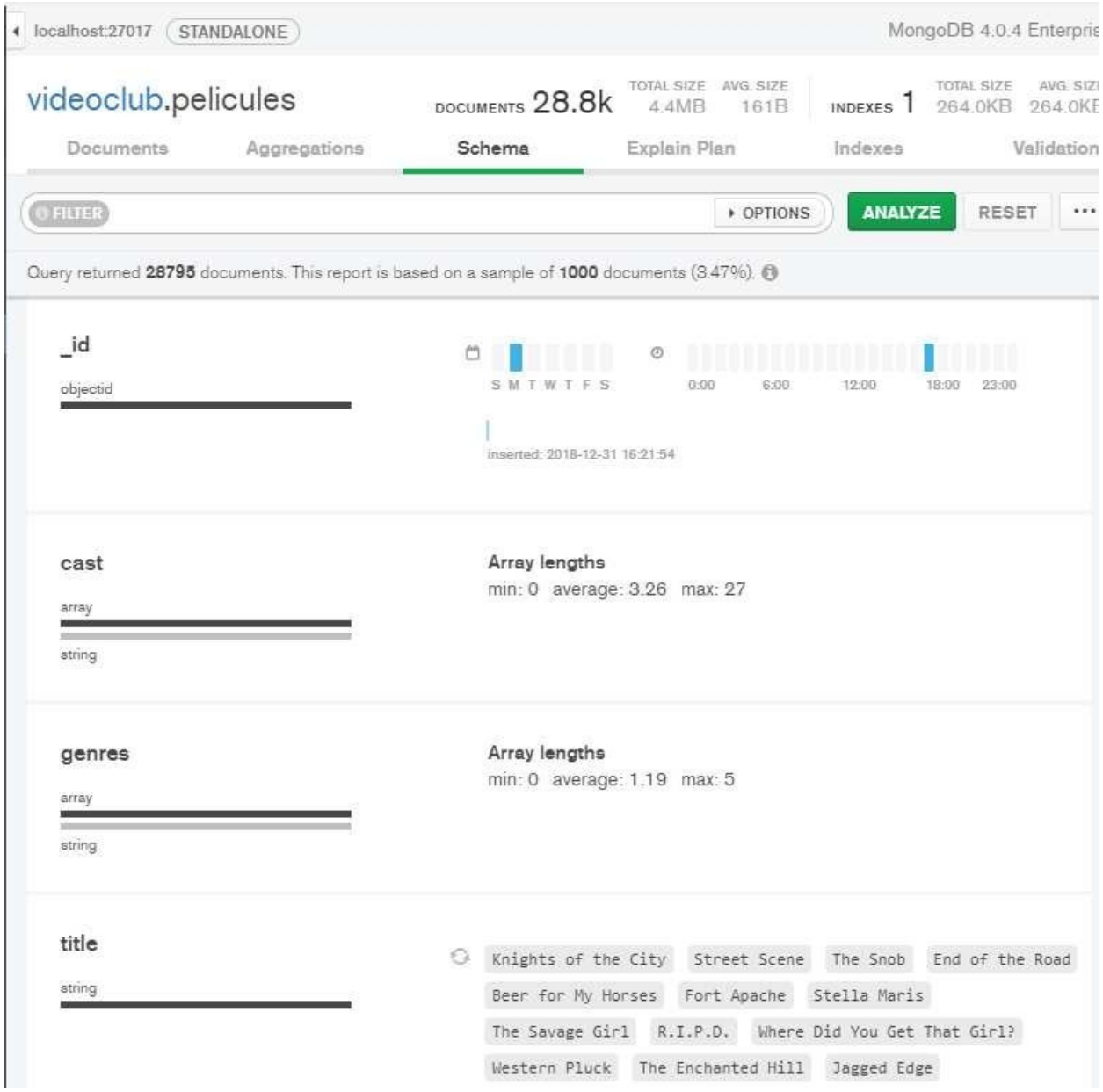
Com podem vore, després d'executar l'ordre "**mongoimport**", ens indica que s'ha connectat al servidor *localhost*, i que ha importat *28.795 documents*.

Si tornem al MongoDB Compass i refresquem la vista de la col·lecció "*pelicules*", vegem que ja tenim tots els documents carregats. Ací es pot vore una vista de la pestanya "**Documents**" on podem explorar tots els documents que hi ha a la base de dades.



Si ens desplacem a la pestanya "**Schema**" podrem vore un anàlisi de l'estructura dels documents que conformen la nostra base de dades.

Ací podrem vore quins són els tipus de dades de cadascun dels camps, el número d'elements que tenen els vectors de dades (arrays), i moltes altres dades interessants sobre esta col·lecció de dades.



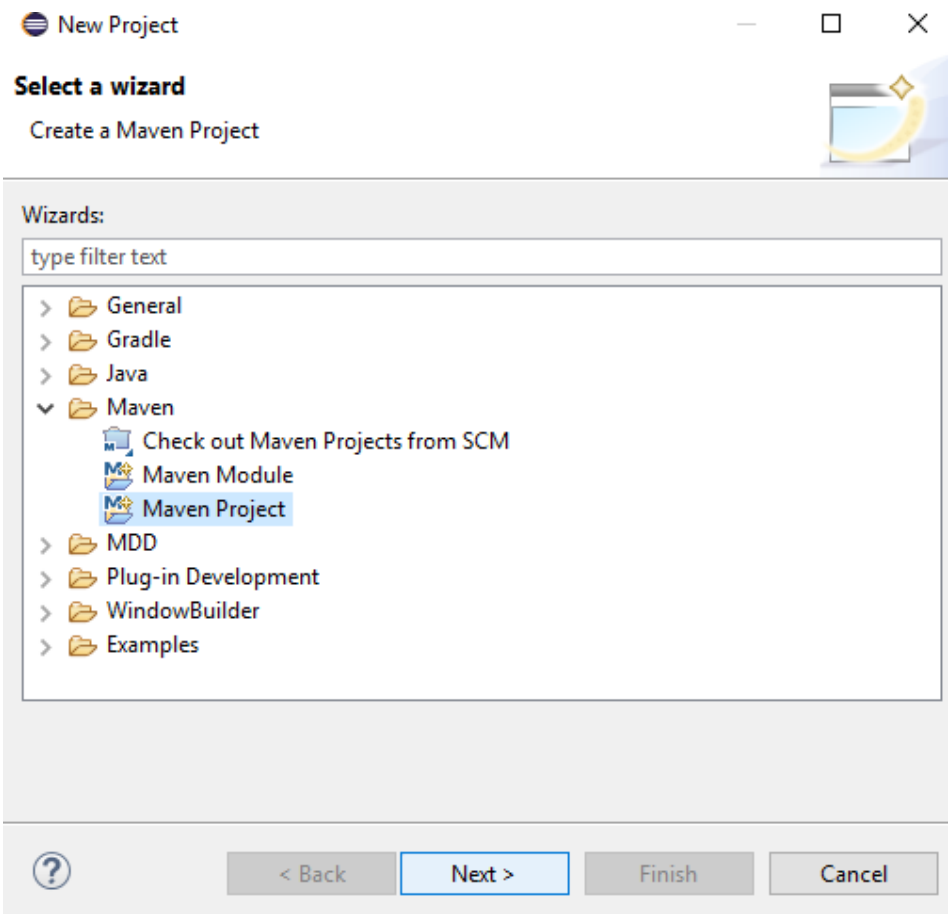
4 Connexió a MongoDB desde Java

4.1 Creació i configuració projecte Java amb connexió amb MongoDB

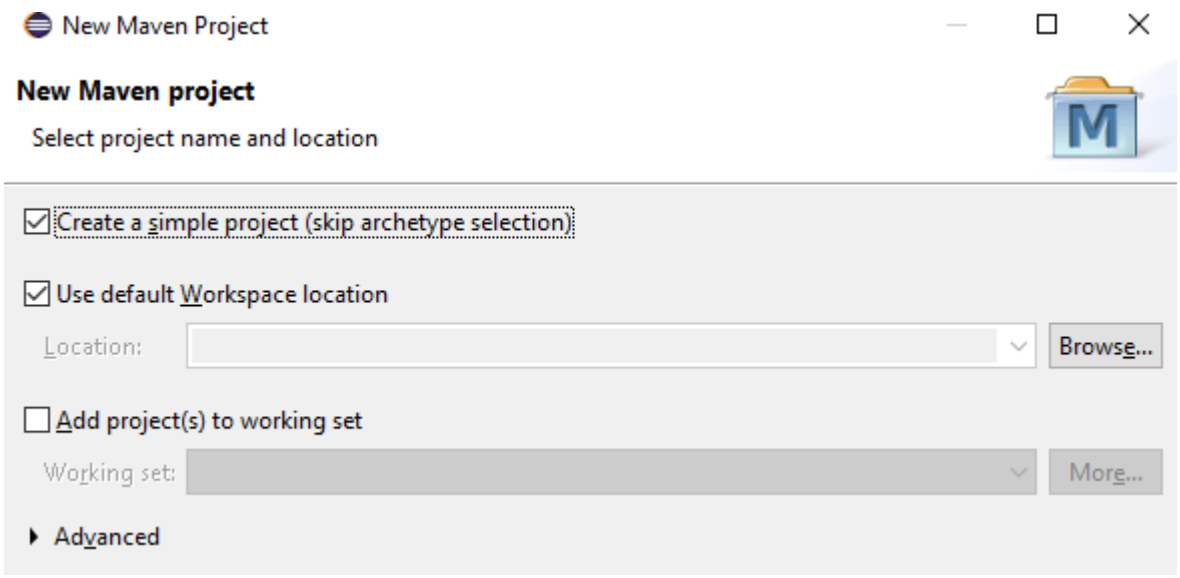
La forma recomanada de començar a utilitzar un dels drivers del projecte és amb un sistema de gestió de dependències. En el nostre cas, utilitzarem el gestor vist en la unitat 3: Maven.

Per començar hem de crear un projecte Maven:

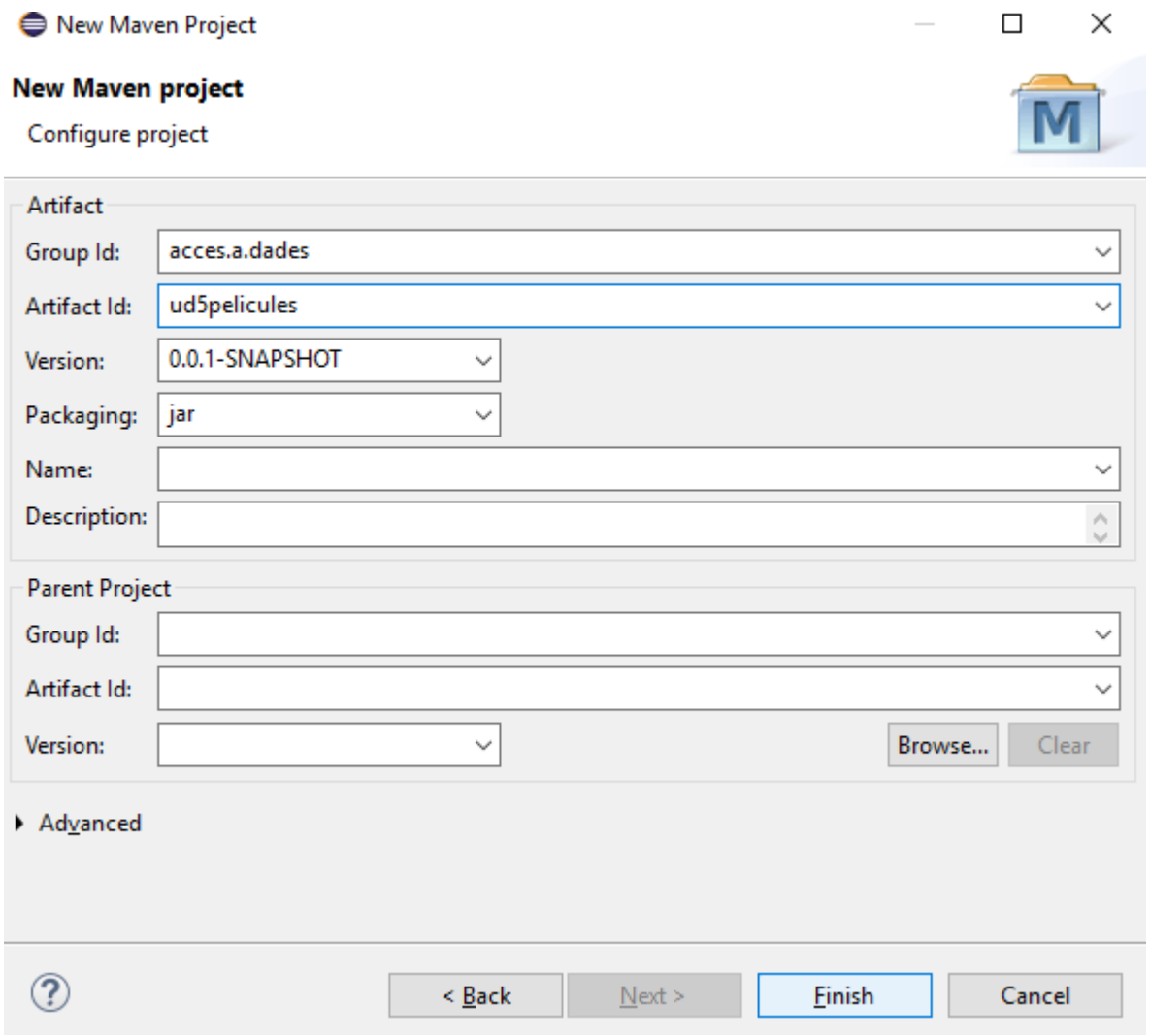
1.- File New Project Maven Maven Project



2.- Marquem les opcions que apareixen a la següent captura.



3.- Posem les següents dades en el formulari i polsem “Finish”.



4.- Ara cal configurar el fitxer de configuració de Maven, "**pom.xml**"

Per a fer-ho, hem d'afegir el següent codi XML al ja existent:

```
<dependencies>
    <dependency>
        <groupId>org.mongodb</groupId>
        <artifactId>mongo-java-driver</artifactId>
        <version>3.12.7</version>
    </dependency>
</dependencies>
```

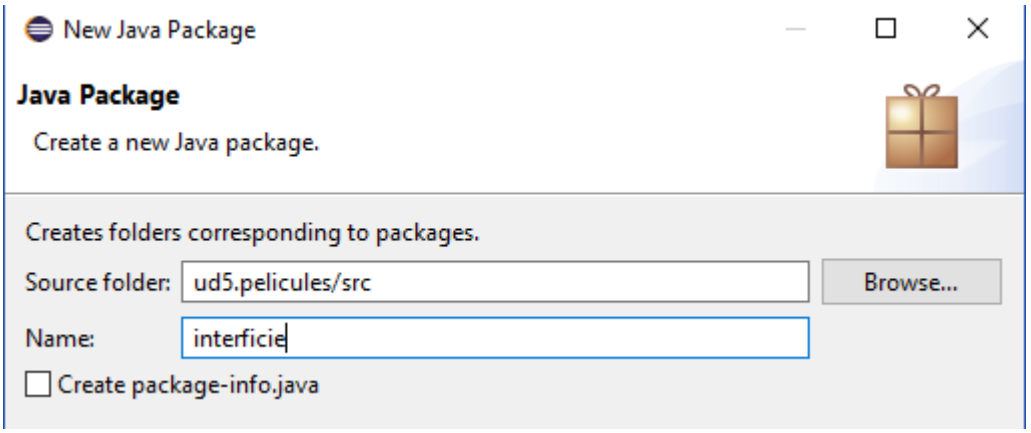
Així, el fitxer **pom.xml** quedarà de la següent manera:

pom.xml

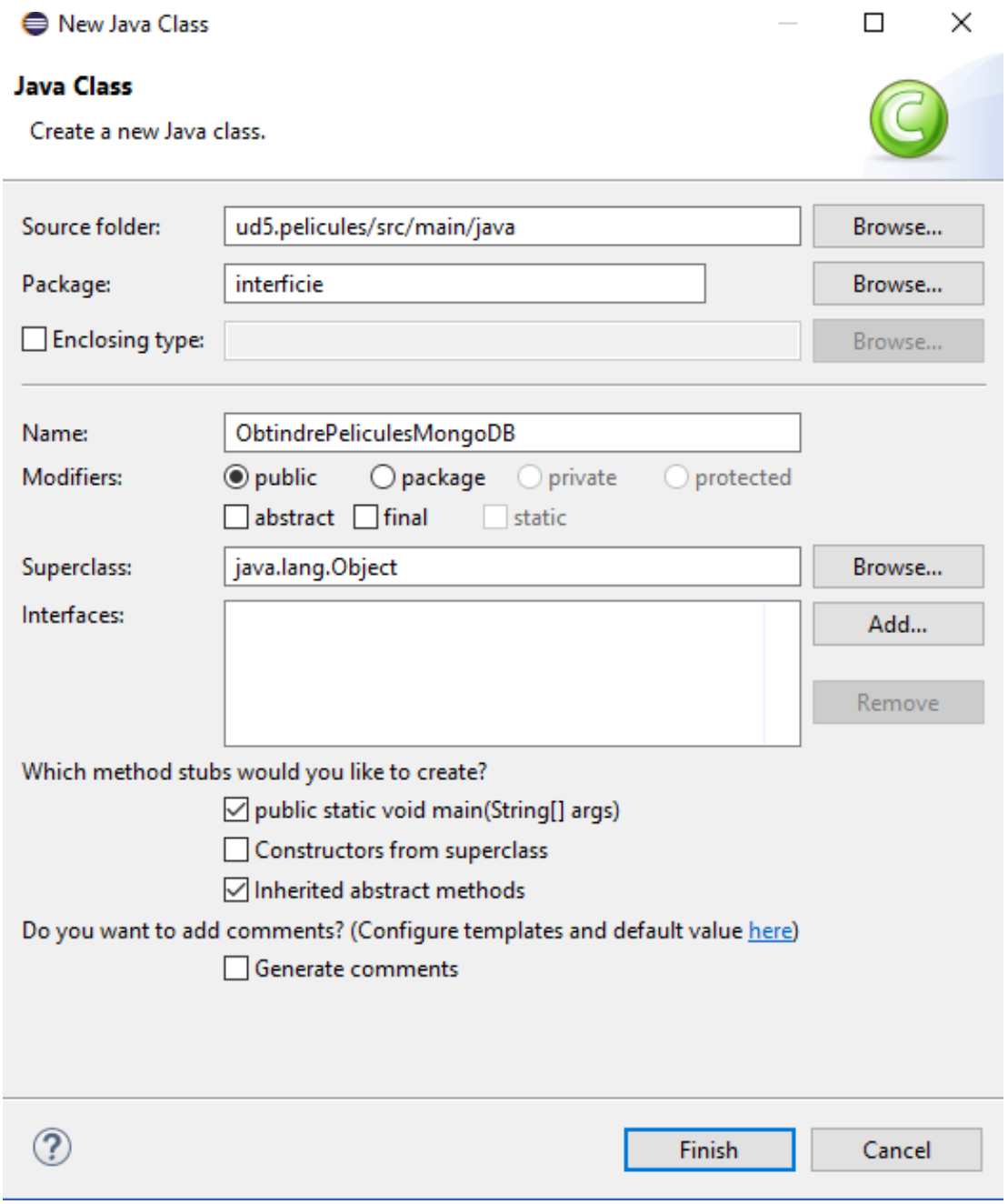
```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns
    <modelVersion>4.0.0</modelVersion>
    <groupId>acces.a.dades</groupId>
    <artifactId>ud5.pelicules</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <dependencies>
        <dependency>
            <groupId>org.mongodb</groupId>
            <artifactId>mongo-java-driver</artifactId>
            <version>3.12.7</version>
        </dependency>
    </dependencies>
</project>
```

4.2 Creem la classe "ObtindrePeliculesMongoDB"

1.- Dins la carpeta "src", creem un paquet "interficie".



2.- En este paquet creem una classe pública **ObtindrePeliculesMongoDB.java** que tinga un mètode **main**.



El resultat serà el següent:

ObtindrePeliculesMongoDB.java

```java
package interficie;

public class ObtindrePeliculesMongoDB {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

    }

}

```

4.2.1 Llibreries de classes per a importar

Per poder utilitzar totes les funcionalitats del controlador de MongoDB, cal importar les següents llibreries:

```java
package interficie;

//NOVA API MongoClient (des de la versió 3.7)
import com.mongodb.ConnectionString;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoClient;

//ELEMENTS COMUNS A TOTES LES API'S de MongoClient

import com.mongodb.ServerAddress;

import com.mongodb.client.MongoDatabase;
import com.mongodb.client.MongoCollection;

import org.bson.Document;
import java.util.Arrays;
import com.mongodb.Block;

import com.mongodb.client.MongoCursor;
import static com.mongodb.client.model.Filters.*;
import com.mongodb.client.result.DeleteResult;
import static com.mongodb.client.model.Updates.*;
import com.mongodb.client.result.UpdateResult;
import java.util.ArrayList;
import java.util.List;
```

4.3 Establint una connexió

Cal utilitzar ***MongoClients.create()*** per a establir una connexió amb una instància de MongoDB en execució.

La instància ***MongoClients*** representa un conjunt de connexions a la base de dades; només es necessitarà una instància de la classe **MongoClient** fins i tot amb múltiples subprocesos.

IMPORTANT:

En general, només es crea una instància de MongoClient per a una implementació de MongoDB determinada (per exemple, standalone, replica set o un clúster fragmentat) i s'utilitza en tota l'aplicació.

No obstant això, si es necessitaren múltiples instàncies:

- Tots els límits d'ús de recursos (per exemple, connexions màximes, etc.) s'apliquen per instància de MongoClient.
- Per a eliminar una instància, cal cridar a MongoClient.close() per a netejar els recursos.

4.3.1 Connectar-se a una única instància de MongoDB

Existixen diverses maneres de connectar-se a un únic servidor MongoDB.

1. Es pot instanciar un objecte MongoClient **sense cap paràmetre** per a connectar-se a una instància MongoDB que s'execute en localhost en el port 27017:

```
public static void main(String[] args) {

    MongoClient mongoClient = MongoClients.create();

}
```

2. Es pot especificar explícitament el **nom d'host** que es connectarà a una instància de MongoDB que s'execute en l'host especificat en el **port 27017**:

```
public static void main(String[] args) {

    MongoClient mongoClient = MongoClients.create(
            MongoClientSettings.builder()
                    .applyToClusterSettings(builder ->
                    builder.hosts(Arrays.asList(new ServerAddress("localhost"))))
                    .build());

}
```

3. Es pot especificar explícitament el **nom d'host i el port**:

```
public static void main(String[] args) {

    MongoClient mongoClient = MongoClients.create(
            MongoClientSettings.builder()
                    .applyToClusterSettings(builder ->
                    builder.hosts(Arrays.asList(new ServerAddress("localhost",27018))))
                    .build());

}
```

4. Es pot especificar la **ConnectionString**:

```
public static void main(String[] args) {

    MongoClient mongoClient = MongoClients.create("mongodb://localhost:27017");

}
```

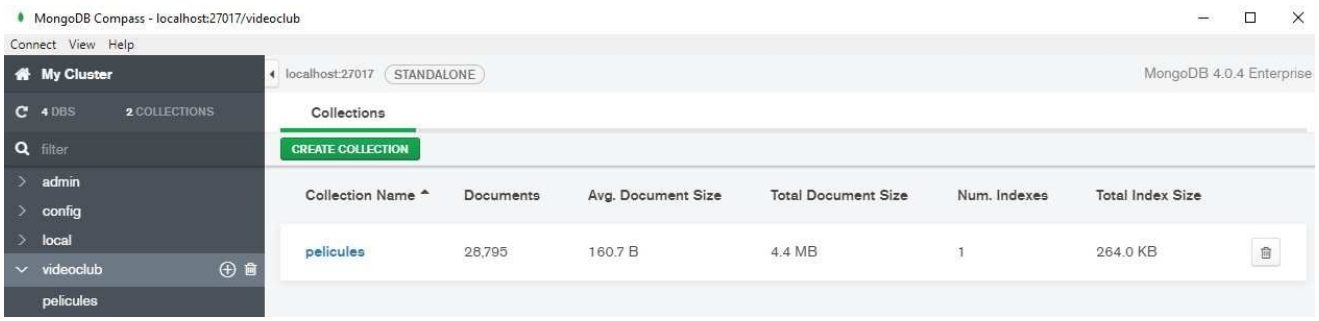
La cadena de connexió segueix principalment el **RFC 3986**, amb l'excepció del nom de domini. Per a MongoDB, és possible llistar múltiples noms de domini separats per una coma.

4.4 Obtenint una Base de dades

Una volta tenim una instància de MongoClient connectada a la base de dades MongoDB, s'ha d'utilitzar el mètode ***MongoClient.getDatabase()***.

Si no existeix una base de dades, MongoDB crea la base de dades quan emmagatzemem per primera vegada les dades per a eixa base de dades.

Anem a recordar quina era la base de dades que hem creat al nostre servidor MongoDB:



Al servidor hem creat la base de dades "videoclub". Per obtindre-la des de la nostra aplicació, utilitzarem el següent codi:

```
public static void main(String[] args) {

    MongoClient mongoClient = MongoClients.create("mongodb://localhost:27017");

    MongoDatabase database = mongoClient.getDatabase("videoclub");

}
```

Les instàncies de ***MongoDatabase*** són immutables.

4.5 Obtenint una Col·lecció

Ara ja tenim una MongoDatabase, i podem accedir a les seues Col·leccions amb el mètode **getCollection()**.

Cal especificar el nom de la col·lecció al cridar al mètode getCollection(). Si no existeix la col·lecció, MongoDB la crea quan emmagatzemem dades en ella per primera volta.

```
public static void main(String[] args) {

    MongoClient mongoClient = MongoClients.create("mongodb://localhost:27017");

    MongoDatabase basedades = mongoClient.getDatabase("videoclub");

    MongoCollection coleccio = basedades.getCollection("pelicules");

}
```

5 Consultes a la base de dades MongoDB

Per a consultar una col·lecció, es pot utilitzar el mètode **find()** de la col·lecció. Podem cridar al mètode sense cap argument per a consultar tots els documents d'una col·lecció o passar un filtre per a buscar documents que coincidisquen amb els criteris de filtre.

El mètode **find()** retorna una instància **FindIterable()** que proporciona una interfície fluida per a encadenar altres mètodes.

5.1 Classe Document

En la interacció amb MongoDB usant el driver de Java se sol utilitzar per a quasi tot la classe **Document**. Utilitzarem esta classe no sols per als documents que hàgem d'introduir, sinó per a definir els criteris d'eliminació, actualització, ordenació i altres.

El **constructor** de la classe Document admet dos paràmetres: una **clau** i un **valor**. Per a crear documents amb més d'una parella clau-valor aplicarem el mètode **append**() al constructor tantes vegades com siga necessari encadenant les crides amb un punt (.). Esta serà la tècnica bàsica per a construir les llistes JSON.

Exemple 1 des de Java:

```
List<String> Hobbies = new ArrayList <String>();

Hobbies.add("musica");
Hobbies.add("pelicules");
Hobbies.add("senderisme");

Document document = new Document ("nom", "Javi").
    append("email", "javi@isca.es").
    append("twitter", "iscajavi").
    append("entreteniments", Hobbies).
    append("location", new Document ("ciutat", "alberic").
                                append("zip", 46260));
```

Exemple 2: Obtindre valors per al document des d'un Pojo:

```
Document documentNou = new Document ()
        . append("titol", llibre. getTitulo())
        . append("descripcio", llibre.getDescripcio())
        . append("autor", llibre. getAutor())
        . append("data", llibre. getData())
        . append("disponible", llibre.getDisponible());
```

5.2 Obtindre el primer document d'una col·lecció

Per a obtindre el primer document de la col·lecció, utilitzem el mètode **find()** sense cap paràmetre i li encadenem el mètode **first()**.

Si la col·lecció està buida, l'operació retorna *null*.

```
public static void main(String[] args) {

    MongoClient mongoClient = MongoClients.create("mongodb://localhost:27017");

    MongoDatabase basedades = mongoClient.getDatabase("videoclub");

    MongoCollection<Document> coleccio = basedades.getCollection("pelicules");

    Document primeraPelicula = coleccio.find().first();

    System.out.println(primeraPelicula.toJson());


}
```

Este exemple imprimeix per consola el següent document:

```
{ "_id" : { "$oid" : "5c2a4222lfe649ebdceacc05" },
 "title" : "Caught",
 "year" : 1900,
 "cast" : [],
 "genres" : [] }
```

Com podem apreciar, a més de la informació pròpia del document, també apareix l'element ***_id*** que afegix MongoDB automàticament per fer referència interna a eixe document.

NOTA:

MongoDB reserva els noms de camp que comencen amb "_" i "$" per a ús intern.

5.3 Obtindre tots els documents d'una col·lecció

Per a obtindre tots els documents de la col·lecció, utilitzarem el mètode **find()** sense cap paràmetre.

Per a recórrer els resultats, hem d'encadenar el mètode **iterator()** al **find()**.

En el següent exemple recuperem tots els documents de la col·lecció i imprimix els documents retornats (28.795 documents):

```
MongoCursor<Document> cursor = coleccio.find().iterator();
try {
    while (cursor.hasNext()) {
        System.out.println(cursor.next().toJson());
    }
} finally {
    cursor.close();
}
```

Encara que es permet el següent codi per a la iteració, **CAL EVITAR** el seu ús ja que l'aplicació pot perdre un cursor si el bucle acaba abans d'hora:

```
for (Document cur : coleccio.find()) {
    System.out.println(cur.toJson());
}
```

5.4 Classe Filters: especificar un filtre de consulta

Per a buscar documents que complisquen certes condicions, passada un objecte de filtre al mètode find(). Per a facilitar la creació d'objectes de filtre, el controlador Java proporciona l'assistent de Filters.

Comparació

PREDICAT	DESCRIPCIÓ
eq	Igual
gt	Mayor que
gte	Mayor o igual
lt	Menor
lte	Menor o igual
ne	No es igual
in	Busca algún valor
nin	Ningún dels valors de una matriu (array)

Matrius (arrays)

OPERADOR	DESCRIPCIÓ
all	Busca matrius (arrays) que contenen els elements especificats en la consulta.
elemMatch	Selecciona tots els documents si un element de la matriu (array) coincidix amb tot el que hem especificat en $elemMatch.
size	Selecciona tots els documents si el camp de la matriu (array) es un tamaño especifico.

Lògics

OPERADOR	DESCRIPCIÓ
and	I
or	O
not	No
nor	Selecciona tots els documents que no cumplixen les condicions especificades.

Exemples

- Buscar els documents en els que el país es “finlandia”:

```
Bson filter =Filters.eq("pais","finlandia");
```

- **AND** Busca per la condició logo = “” y poblacio=”Helsinki”:

```
Bson filter =Filters.and(
   Filters.eq("logo",""), Filters.eq("poblacio",”Helsinki”)
);//AND
```

- **OR** Busca documents amb logo =”” o codi=8579:

```
Bson filter =Filters.or(
   Filters.eq("logo",""),
   Filters.eq("codi",8579)
   ); //OR
```

- Busca Codi=8579 **o** Població=”Espoo” **o** País = “Espanya” **o** Sigles = “fi”

```
Bson filter =Filters.and(
   Filters.or(
        Filters.eq("Codi", 8579),
        Filters.eq("Poblacio",”Espoo”),
        Filters.or(
             Filters.eq("Pais", "Espanya"),
             Filters.eq("Sigles", "fi")
        )//or
   ) //or
);//and
```

Exemples matrius (arrays)

- Selecciona tots els documents amb una matriu (array) "Provincies" que continga la província "Alacant":

```
Bson filter = Filters.all("Provincies", "Alacant"));
```

- Selecciona tots els documents amb una matriu (array) "Provincies" que continga les províncies "Alacant" y "Valencia":

```
Bson filter = Filters.all("Provincies",
  Arrays.asList("Valencia", "Alacant")
);
```

5.4.1 Obtindre un document únic que coincidisca amb un filtre

Per exemple, per a trobar el primer document on el camp "year" té el valor "2017", li passem un objecte de filtre ***eq*** per a especificar la condició d'igualtat:

```
package interficie;

import org.bson.Document;
import org.bson.conversions.Bson;

import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import static com.mongodb.client.model.Filters.*;

public class ConsultaPelicules {

    public static void main(String[] args) {

        MongoClient mongoClient = MongoClients.create("mongodb://localhost:27017");

        MongoDatabase basedades = mongoClient.getDatabase("videoclub");

        MongoCollection<Document> coleccio = basedades.getCollection("pelicules");

        //5.1 Obtindre 1 pelicula de l'any 2017
        Document pelicula2017 =  coleccio.find(eq("year", 2017)).first();

        System.out.println(pelicula2017.toJson());



    }
```

Este codi ens imprimirà el següent document:

```
{ "_id" : { "$oid" : "5c2a4222lfe649ebdceb3ac2" },
"title" : "Monster Trucks",
"year" : 2017,
"cast" : ["Lucas Till", "Jane Levy", "Amy Ryan", "Holt McCallany", "Rob Lowe",
"Danny Glover", "Frank Whaley", "Chad Willett", "Barry Pepper", "Thomas Lennon", "Tucker Albrizzi"],
"genres" : ["Animated", "Adventure", "Science Fiction"] }
```

5.4.2 Obtindre tots els documents que coincideixen amb un filtre

Exemple 1:

El següent exemple retorna i imprimeix totes les pel·lícules que s'han llançat al mercat després de 2017:

```
//5.3.2 Obtindre totes les pel.lícules des de 2017 fins ara
Block<Document> printBlock = new Block<Document>() {
     @Override
     public void apply(final Document document) {
         System.out.println(document.toJson());
     }
};

coleccio.find(gt("year", 2017)).forEach(printBlock);
```

L'exemple utilitza el mètode ***forEach*** en l'objecte ***FindIterable*** per a aplicar un bloc a cada document.

5.5 Contar documents en una col·lecció

Per a contar el nombre de documents d'una col·lecció, es pot utilitzar el mètode ***countDocuments()*** de la col·lecció.

El següent codi ha d'imprimir 28.975, que són els documents que té la col·lecció "pelicules".

```
// 5.4 Contar els documents que té la col.lecció "pelicules"

System.out.println("La col.lecció \"pelicules\" té "+coleccio.countDocuments());
```

5.6 Buscar patrons en els camps dels documents

Per buscar una expressió regular en un camp, s'utilitza l'operador **regex**. Per crear els patrons, a més del text, podem utilitzar caràcters comodí:

^Mary * Vol dir que comence per "Mary " i després que tinga els caràcters que vullga.

. Vol dir un caràcter.

* Vol dir cap o molts caràcters.

*** Poppins$** Vol dir que el text acaba en " Poppins".

Exemple 3:

Extrau totes les pel·lícules que tenen el text “Mary Poppins” en el camp “title”:

```
//5.5 Buscar el patró "Mary Poppins" en el camp "title" dels documents
Block<Document> printBlock = new Block<Document>() {
    @Override
    public void apply(final Document document) {
        System.out.println(document.toJson());
    }
};

coleccio.find(regex("title","Mary Poppins")).forEach(printBlock);
```

Exemple 4:

Busca totes les pel·lícules que comencen per “Mary ” en el camp “title” i imprimix quantes hi ha. Este exemple ha de tornar 16 pel·lícules.

```
//5.5.2 Buscar les pelicules que comencen per "Mary " en el camp "title"
//i dir quantes hi ha

long numDocuments = coleccio.countDocuments(Filters.regex("title","^Mary *"));
System.out.println("Ha trobat "+numDocuments+" películes.");
```

Exemple 5:

Extrau totes les pel·lícules que acaben en “ Poppins” en el camp “title”:

```
//5.5.3 Buscar les pelicules que acaben per " Poppins" en el camp "title"
Block<Document> printBlock = new Block<Document>() {
    @Override
    public void apply(final Document document) {
        System.out.println(document.toJson());
    }
};

coleccio.find(regex("title"," Poppins$")).forEach(printBlock);
```

5.7 Consultes sobre arrays

5.7.1 Buscar elements dins d’un array

Per buscar si un element es troba dins d’un camp de tipus Array, utilitzarem l’operador **$all**.

Exemple 6:

Extrau totes les pel·lícules en les que ha participat l’actor “Danny Glover”:

```
// 5.6.1 Buscar les pelicules que en les que ha participat l'actor "Danny Glover"
Block<Document> printBlock = new Block<Document>() {
    @Override
    public void apply(final Document document) {
        System.out.println(document.toJson());
    }
};

coleccio.find(all("cast", "Danny Glover")).forEach(printBlock);
```

5.7.2 Contar els elements que conté un array

Per saber el número d'elements que té un camp de tipus array, utilitzarem l'operador **$size**.

Exemple 7:

Extrau totes les pel·lícules en les que han participat 6 actors:

```
// 5.6.2 Buscar les pelicules que en les que han participat 6 actors
Block<Document> printBlock = new Block<Document>() {
    @Override
    public void apply(final Document document) {
        System.out.println(document.toJson());
    }
};

coleccio.find(size("cast", 6)).forEach(printBlock);
```

5.8 Projeccions

Al realitzar una consulta i imprimim tot el documents, ens apareixen tots els camps que té cada document. Tal volta únicament estem interessats en algun d'estos camps.

Quan volem que sols apareguen una part dels camps dels documents resultants d'una consulta, el que necessitem és fer una **projecció** dels resultat.

En MongoDB açò es pot fer de dues maneres:

1. **Especificant literalment com volem que siga el document resultant**:

```
//5.7.1 Mostra el títol i els actors de totes les comèdies de l'any 2000
Block<Document> printBlock = new Block<Document>() {
    @Override
    public void apply(final Document document) {
        System.out.println(document.toJson());
    }
};

coleccio.find(and(eq("year", 2000),all("genres", "Comedy")))
.projection(new Document("title", 1)
        .append("cast",1)
        .append("_id", 0))
.forEach(printBlock);
```

2. Utilitzant la classe **Projections** que proporciona el driver de MongoDB. D'esta manera indicarem quins camps volem que apareguen en els documents resultants de la consulta d'una manera més intuïtiva.

```
// 5.7.2 Mostra el títol i els actors de totes les comèdies de l'any 2000
Block<Document> printBlock = new Block<Document>() {
    @Override
    public void apply(final Document document) {
        System.out.println(document.toJson());
    }
};

coleccio.find(and(eq("year", 2000), all("genres", "Comedy")))
        .projection(fields(include("title", "cast"), excludeId()))
        .forEach(printBlock);
```

5.9 Ordenant els resultats

Si volem ordenar documents, cal passar-li un *document d'especificació d'ordenació* al mètode **FindIterable.sort()**. El controlador Java proporciona **Sorts** *helpers*, per facilitar el document d'especificació de la classificació.

Exemple 8:

Extrau el títol i els actors de totes les comèdies de l'any 2008 ordenades alfabèticament:

```
// 5.8.1 Mostra el títol i els actors de totes les comèdies de l'any 2008
// ordenades alfabèticament
Block<Document> printBlock = new Block<Document>() {
    @Override
    public void apply(final Document document) {
        System.out.println(document.toJson());
    }
};

coleccio.find(and(eq("year", 2008), all("genres", "Comedy")))
        .sort(Sorts.ascending("title"))
        .projection(fields(include("title", "cast"), excludeId())).forEach(printBlock);
```

6 Inserció de documents a la base de dades MongoDB

Per a este apartat, anem a crear una base de dades de prova anomenada "*software*" i dins d'ella, una col·lecció "*aplicacions*".

```
public static void main(String[] args) {

    MongoClient mongoClient = MongoClients.create("mongodb://localhost:27017");

    MongoDatabase basedades = mongoClient.getDatabase("software");

    MongoCollection<Document> coleccio = basedades.getCollection("aplicacions");
```

6.1 Crear un document

Per a crear el document utilitzant el controlador Java, utilitzem la classe **Document**. Per exemple, considerem el següent document de JSON:

```
{
 "nom": "MongoDB",
 "tipus" : "base de dades",
 "conte": 1,
 "versions": "v3.2", "v3.0", "v2.6" ],
 "info": { x : 203, i : 102 }
}
```

Per a crear el document utilitzant el controlador Java, s'ha d'instanciar un objecte **Document** amb un camp i un valor, i utilitzar el seu mètode **append()** per a incloure camps i valors addicionals a l'objecte document. El valor pot ser un altre objecte Document per a especificar un document incrustat:

```
//6.1 Crear Documents
 Document doc = new Document("nom", "MongoDB")
            .append("tipus", "database")
            .append("conte", 1)
            .append("versions", Arrays.asList("v3.2", "v3.0", "v2.6"))
            .append("info", new Document("x", 203).append("y", 102));
```

NOTA:

El tipus d'array BSON correspon al tipus Java java.util.List.

6.2 Inserir un Document

Una vegada que tenim l'objecte **MongoCollection**, es pot inserir documents en la col·lecció. Per inserir un sol document en la col·lecció, es pot utilitzar el mètode **insertOne()** de classe ***Collection***.

```
//6.2.1 Inserir un document
coleccio.insertOne(doc);
```

NOTA:

Tal i com hem vist a l'apartat 5.1, si no s'especifica cap camp **_id** de nivell superior en el document, MongoDB afig automàticament el camp **_id** al document inserit.

6.3 Inserir Múltiples Documents

Per a afegir múltiples documents, es pot utilitzar el mètode **insertMany**() de la classe ***Collection*** que pren una llista de documents per a inserir.

El següent exemple agregarà múltiples documents de la forma:

```
{ "contador" : valor }
```

Creem els documents en un bucle i els afegim a una llista de documents:

```
//6.2.2 Inserir molts documents
List<Document> documents = new ArrayList<Document>();
for (int valor = 0; valor < 100; valor++) {
    documents.add(new Document("contador", valor));
}
```

Per inserir estos documents en la col·lecció, li passem la llista de documents al mètode **insertMany**():

```
coleccio.insertMany(documents);
```

Una volta inserits, podem comprovar que en la nova col·lecció que hem creat a la base de dades, s'han creat realment. Una forma de vore-ho ràpidament seria consultant el número de documents que té la col·lecció.

```
System.out.println("La col.lecció té "+coleccio.countDocuments()+" documents.");
```

7 Actualització de documents a la base de dades MongoDB

Per a este apartat, anem a continuar amb la base de dades de prova creada en l'apartat anterior.

Si volem actualitzar els documents d'una col·lecció, es poden utilitzar els mètodes **updateOne** i **updateMany** de classe ***Collection***.

Cal passar als mètodes:

- Un **objecte de filtre** per a determinar el document o els documents a actualitzar. Per a facilitar la creació d'objectes de filtre, el controlador Java proporciona el **Filters** helper. Per a especificar un filtre buit (és a dir, que coincidisca amb tots els documents d'una col·lecció), utilitzarem un objecte **Document** buit.
- Un **document d'actualització** que especifica les modificacions. Per a obtindre una llista dels operadors disponibles, consulte Actualitzar operadors.

Els mètodes d'actualització retornen un **UpdateResult** que proporciona informació sobre l'operació, incloent el nombre de documents modificats per l'actualització.

7.1 Actualitzar un únic document

Per tal d'actualitzar com a molt un document, utilitzem el mètode **updateOne**.

En el següent exemple s'actualitza el primer document que complisca la condició del filtre "contador" és igual a **10** i li canvia el valor de "contador" a **110**.

```
MongoClient mongoClient = MongoClients.create("mongodb://localhost:27017");

MongoDatabase basedades = mongoClient.getDatabase("software");

MongoCollection<Document> coleccio = basedades.getCollection("aplicacions");

// 7.1 UpdateOne: Modifica el valor del camp "contador" en aquell document que valga 10
coleccio.updateOne(eq("contador", 10), new Document("$set", new Document("contador", 110)));
```

7.2 Actualitzar múltiples documents

Per tal d'actualitzar tots els documents que complisquen la condició del filtre, utilitzem el mètode **updateMany**.

En el següent exemple s'incrementa el valor de "contador" en 50, en tots els documents on "contador" és menor que **80**.

```
//7.2 UpdateMany: Incrementa el valor de "contador" en 50,
//en tots els documents on "contador" és menor que 80
UpdateResult updateResult = coleccio.updateMany(lt("contador", 80), inc("contador", 50));
System.out.println(updateResult.getModifiedCount());
```

8 Eliminació de documents a la base de dades MongoDB

Continuem amb la base de dades "software" creada en l'apartat 6.

Per tal de borrar documents d'una col·lecció, es poden utilitzar els mètodes **deleteOne** i **deleteMany** de classe ***Collection***.

Cal passar als mètodes un objecte de filtre per a determinar el document o documents a esborrar. Per a especificar un filtre buit (és a dir, que coincidisca amb tots els documents d'una col·lecció), utilitzem un objecte **Document** buit.

Els mètodes d'eliminació retornen un **DeleteResult** que proporciona informació sobre l'operació, incloent el nombre de documents esborrats.

8.1 Eliminar un únic document que compleix una condició

Per tal d'eliminar com a molt un document que complisca la condició del filtre, utilitzem el mètode **deleteOne**.

En el següent exemple s'esborra el primer document que complisca la condició del filtre "contador" és igual a **110.**

```
// 8.1 DeleteOne: Elimina el document que tinga 110 en el camp "contador".
coleccio.deleteOne(eq("contador", 110));
```

8.2 Eliminar tots els documents que compleixen una condició

Per tal d'esborrar tots els documents que complisquen la condició del filtre, utilitzem el mètode **deleteMany**.

En el següent exemple s'esborren tots els documents on el valor de "contador" es major o igual a 50.

```
// 8.2 DeleteMany: Elimina tots els documents on "contador" és major o igual a 50
DeleteResult deleteResult = coleccio.deleteMany(gte("contador", 50));
System.out.println(deleteResult.getDeletedCount());
```